

9. Číselné soustavy a převody. Kódování čísel s pevnou a pohyblivou řádovou tečkou. Kódování záporných čísel.

A. Číselné soustavy a převody

Čísla v poziční soustavě o základu b :
$$(a_n \dots a_0 . a_{-1} \dots a_{-m})_b = \sum_{i=-m}^n a_i b^i$$

Základní soustavy: * **Binární** ($b = 2$): Cifry 0, 1. Základ pro digitální systémy. *

Oktalová ($b = 8$): Cifry 0-7. Historicky pro kompaktní binární zápis (3 bity = 1 cifra). *

Decimální ($b = 10$): Cifry 0-9. Pro lidi, vstup/výstup. * **Hexadecimální** ($b = 16$): Cifry 0-9, A-F (A=10, ..., F=15). 1 cifra = 4 bity. Pro adresy, barvy, kompaktní binární data.

Převody mezi soustavami

Z libovolné do desítkové: Polynomický rozvoj.

$$(1101)_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (13)_{10}$$

Z desítkové do libovolné:

Celá část: Postupné dělení základem, zbytky zdola nahoru.

Desetinná část: Postupné násobení základem, celé části shora dolů.

Mezi binární a hexadecimální: Skupiny po 4 bitech od řádové tečky.

$$(11001011)_2 \rightarrow (1100)_2 (1011)_2 \rightarrow (CB)_{16}$$

B. Kódování záporných čísel

Pro celá čísla (integers) se používají:

Přímý kód (Sign-and-Magnitude):

Princip: Nejvyšší bit = znaménko (0=+, 1=-). Zbytek bitů = absolutní hodnota.

Nevýhody: Dvě nuly (+0, -0), složitá aritmetika.

Inverzní kód (Ones' Complement):

Princip: Záporné číslo = bitová inverze kladného čísla.

Nevýhody: Dvě nuly (+0, -0), složitější aritmetika (end-around carry).

Doplňkový kód (Two's Complement) – Standard v moderních CPU:

Princip: Záporné číslo = (bitová inverze kladného čísla) + 1.

Příklad (-3 pro 4 bity): $(0011)_2 \xrightarrow{\text{invert}} (1100)_2 \xrightarrow{+1} (1101)_2$

Výhody:

Jedna unikátní nula: Pouze $(0000)_2$.

Jednotný hardware pro aritmetiku: Sčítání/odčítání funguje stejně pro kladná i záporná čísla ($A - B = A + (-B)$).

Rozsah pro n bitů: $\langle -2^{n-1}, 2^{n-1} - 1 \rangle$.

C. Čísla s pevnou řádovou tečkou (Fixed-point)

Princip: Pozice řádové tečky je pevně daná. Bity pro celou část, bity pro desetinnou část.

Formát: $QI.F$ (I bitů celá část, F bitů desetinná část).

Výhody: Rychlé výpočty (Integer ALU), konstantní přesnost.

Nevýhody: Omezený dynamický rozsah, nutnost ručního škálování.

Použití: Mikrokontroléry bez FPU, DSP, embedded systémy.

D. Čísla s pohyblivou řádovou tečkou (Floating-point)

Princip: Vědecký zápis, standard IEEE 754.

Hodnota:

$$\text{Hodnota} = (-1)^s \cdot (1 + m) \cdot 2^{e - \text{bias}}$$

s : Znaménko (1 bit).

e : Exponent (kódován s posunutou nulou - bias).

m : Mantisa (zlomková část, skrytá jednička $1.mmm$).

Standardní formáty IEEE 754

Single Precision (float): 32 bitů (1s, 8e, 23m). bias = 127. Rozsah $\approx \pm 10^{38}$, přesnost ≈ 7 desetinných míst.

Double Precision (double): 64 bitů (1s, 11e, 52m). bias = 1023. Rozsah $\approx \pm 10^{308}$, přesnost $\approx 15 - 17$ desetinných míst.

Speciální hodnoty IEEE 754

Nula: Kladná i záporná (exponent i mantisa 0).

Nekonečno ($\pm\infty$): Exponent samé 1, mantisa samé 0.

NaN (Not a Number): Exponent samé 1, mantisa nenulová.

Typické zkouškové otázky

Převody:

$$\$(25)\{10\} = (11001)_2 = (19)\{16\}\$$$

$$\$(1A)\{16\} = (00011010)_2 = (26)\{10\}\$$$

Dvojkový doplněk:

Princip: Kladné číslo invertovat bity a přičíst 1.

Výhody: Jedna nula, jednotná aritmetika (sčítání/odčítání), větší rozsah záporných čísel.

Bit, nibble, byte, word:

Bit: Základní jednotka (0 nebo 1).

Nibble: 4 bity (jedna hexadecimální cifra).

Byte: 8 bitů (nejmenší adresovatelná jednotka).

Word: Skupina bitů zpracovávaná procesorem (velikost dle architektury, např. 32/64 bitů).

10. Programátorský model procesoru (TAHÁK)

Klíčové pojmy

Instrukce, registr, ALU, Program Counter (PC), Stack Pointer (SP), paměť, přerušení, instrukční soubor (ISA), symbolická adresa, fetch-decode-execute cyklus, adresní režimy, konvence volání, zásobníkový rámeček, návratová adresa, obslužná rutina přerušení (ISR), kontext procesu.

10.1 Programátorský model procesoru a registry

Programátorský model procesoru je abstrakce hardwaru viditelná programátorovi (Assembler). Zahrnuje **registry**, **instrukční sadu**, **adresní režimy** a **viditelný stav procesoru**.

Fetch-Decode-Execute cyklus: 1. **Fetch:** Načtení instrukce z paměti (adresa z PC). 2. **Decode:** Dekódování instrukce (operace, operandy). 3. **Execute:** Provedení operace (ALU, přesun dat, skok).

Registry (rychlé paměti v CPU): * **Všeobecné registry (GPR):** Ukládání operandů a mezivýsledků. Např. `$EAX/$RAX`, `$EBX/$RBX` (x86/x64). * **Ukazatel instrukcí (IP/PC):** Adresa následující instrukce. Automaticky inkrementován. * **Ukazatel zásobníku (SP):** Ukazuje na vrchol zásobníku (Stack) v RAM. Pro návratové adresy, lokální proměnné. * **Stavový registr (Flags):** Bity reflektující výsledek operace (ALU). Klíčové: `$ZF` (nula), `$CF` (přenos), `$OF` (přetečení znaménkového), `$SF` (záporný).

10.2 Instrukce a instrukční soubor (ISA)

Instrukční soubor (ISA): Sada všech instrukcí, kterým procesor rozumí (např. x86, ARM). **Strojová instrukce:** Binární slovo. * **Operační znak (Opcode):** Typ operace. * **Operandy:** Data pro operaci (registry, literály, adresy).

Typy operací: * Aritmetické a logické (ALU): `$ADD`, `$SUB`, `$AND`. * Přesuny dat: `$MOV` (registr-registr, registr-paměť). * Skoky a volání podprogramů: `$JMP`, `$CALL`. * I/O operace: Komunikace s periferiemi (např. `$IN`, `$OUT` nebo Memory-Mapped I/O).

10.3 Jazyk symbolických adres (JSA / Assembler)

Assembler: Textové zkratky (mnemoniky) pro strojový kód. **Symbolická adresa (Návěští/Label):** Textový štítek místo číselné adresy (např. `loop_start:`). Překladač

nahradí reálnou adresou. **Algoritmizace:** `$if-else$` , `$while$` , `$for$` se realizují pomocí instrukcí porovnání (`$CMP$`) a podmíněných/nepodmíněných skoků (`$JNE$` , `$JMP$`). Příklad `$if (A == B)$ : $CMP EAX, EBX; JNE rovno_ne; ... (A==B); JMP konec_if; rovno_ne: ... (else); konec_if: ...`

10.4 Časování programu, podprogramy a přerušení

Časování programu

Instrukce trvají určitý počet taktů CPU.

Operace s registry: obvykle 1 takt.

Přístup do paměti: výrazně dražší.

Podprogramy (Funkce)

Využívají **programový zásobník (Stack)**. * `CALL podprogram` : Uloží **návratovou adresu** (aktuální PC) na zásobník, změní PC na adresu podprogramu. * `RET` :

Vyzvedne návratovou adresu ze zásobníku do PC. * **Zásobníkový rámec**: Na zásobníku pro parametry, lokální proměnné, návratovou adresu. * **Konvence volání**: Určují předávání parametrů a úklid zásobníku.

Přerušení (Interrupt)

Mechanismus pro asynchronní přerušení programu (hardware/software). *

Hardwarové: Periferie (klávesa, síť). * **Softwarové (Exception/Trap)**: Instrukce (`$syscall`), chyba (dělení nulou).

Průběh zpracování přerušení: 1. Dokončení aktuální instrukce. 2. Uložení **kontextu** (Flags, PC) na zásobník. 3. Skok na **obslužnou rutinu (ISR)** dle **tabulky vektorů přerušení**. 4. Obsluha události v ISR. 5. Návrat instrukcí `$IRET$` (obnoví kontext ze zásobníku).

Praktické použití

Optimalizace kódu, embedded systémy, ladění chyb paměti, pochopení OS a ovladačů.

Nejčastější chyby u zkoušky

Zaměňování registru a paměťové adresy.

Přerušení není totéž co funkční volání (asynchronní, ukládá více kontextu).

Ignorování uloženého kontextu procesu.

Typické zkouškové otázky

Popište fetch-decode-execute cyklus.

Fetch: Načtení instrukce z paměti (adresa z PC).

Decode: Dekódování instrukce (operace, operandy).

Execute: Provedení operace. PC se aktualizuje.

Jakou roli mají registry PC a SP?

PC (Program Counter): Adresa následující instrukce. Řídí tok programu.

SP (Stack Pointer): Ukazuje na vrchol zásobníku v RAM. Pro návratové adresy, lokální proměnné, parametry.

Co se děje při přerušení?

Procesor dokončí instrukci.

Uloží **kontext** (PC, Flags) na zásobník.

Skočí na **obslužnou rutinu (ISR)** z **tabulky vektorů přerušení**.

ISR obslouží událost.

Instrukce `$IRET$` obnoví kontext ze zásobníku, program pokračuje.

Shrnutí kapitoly

Programátorský model procesoru je základní abstrakce pro nízkoúrovňové programování. Klíčové je pochopení fetch-decode-execute cyklu, rolí registrů (PC, SP, Flags), instrukčního souboru a mechanismů podprogramů a přerušení pro efektivní programování a porozumění OS/hardware.

Jazyk C: Základní datové typy, struktury, pole, ukazatele a dynamická alokace

Klíčové pojmy

Datový typ, Celočíselný typ (`char` , `int` , `long`), Desetinný typ (`float` , `double`), Logický typ (`_Bool`), `void` , `enum` , `struct` , `union` , Pole, Ukazatel, Adresa (`&`), Dereference (`*`), Ukazatelová aritmetika, Dynamická paměť, Zásobník (Stack), Halda (Heap), `malloc` , `calloc` , `realloc` , `free` , Memory leak, Dangling pointer, `NULL` pointer, Padding, Typová konverze (casting).

A. Základní a strukturované datové typy

1. Základní (skalární) datové typy

Uchovávají jednu hodnotu.

Celočíselné:

`char` : 1 bajt (8 bitů), znaky/malá čísla. `signed` (-128 až 127), `unsigned` (0 až 255).

`short` : Min. 16 bitů.

`int` : Min. 16 bitů (často 32 bitů).

`long` : Min. 32 bitů.

`long long` : Min. 64 bitů (C99).

Modifikátory: `signed` (výchozí), `unsigned` .

Desetinné (IEEE 754):

`float` : Jednoduchá přesnost (32 bitů), cca 7 desetinných míst.

`double` : Dvojitá přesnost (64 bitů), cca 15-17 desetinných míst.

`long double` : Rozšířená přesnost (implementačně závislá).

Logický:

C89: `0` = false, nenulová = true.

C99: `_Bool` (S `<stdbool.h>` jako `bool` , `true` , `false`).

`void` : Neúplný typ. Pro funkce bez návratové hodnoty/parametrů, generické ukazatele (`void*`).

Výčtový typ (`enum`): Sada pojmenovaných celočíselných konstant. Zvyšuje čitelnost.

```
c enum Barva { CERVENA, ZELENA, MODRA };
```

2. Strukturované datové typy

Seskupují více proměnných pod jedno jméno.

Struktura (struct):

Seskupuje členy (různých typů) do jednoho celku. Každý člen má vlastní paměť.

Velikost: Součet velikostí členů + případný *padding* (zarovnání).
`c struct Bod { int x, y; }; struct Bod b = {10, 20}; // typedef struct Bod { int x, y; } Bod;`

Unie (union):

Členové sdílejí stejné paměťové místo. Velikost je rovna velikosti největšího člena.

V daný okamžik platí hodnota jen jednoho člena.
`c union Data { int i; float f; }; union Data d; d.i = 5; // d.f je neplatné`

Hlavní rozdíl: `struct` = oddělená paměť pro všechny členy; `union` = sdílená paměť pro všechny členy.

B. Pole (array) a Ukazatele (pointer)

1. Pole

Homogenní, souvislá paměť pro pevný počet prvků stejného typu.

Deklarace: `int pole[5];`

Indexování: Od 0 do N-1 . `pole[0]` .

Jméno pole: Ve většině výrazů se chová jako konstantní ukazatel na první prvek (`&pole[0]`).

Vícerozměrná pole: Pole polí, uloženo po řádcích. `int matice[3][4];`

Pole jako parametry: "Rozpadne se" na ukazatel na první prvek. Velikost je nutné předat zvlášť.

2. Ukazatele (Pointery)

Proměnná, jejíž hodnotou je paměťová adresa.

Deklarace: `int *p;`

Operátory:

& (adresa z): Získá adresu proměnné. `int x = 10; int *p = &x;`

*** (hodnota na adrese / dereference):** Přistoupí k hodnotě na adrese, kam ukazatel ukazuje. `*p = 20;`

NULL ukazatel: Neukazuje na platnou paměť. `int *ptr = NULL;`

void* (generický ukazatel): Může uchovávat adresu jakéhokoli typu. Pro dereferenci/aritmetiku nutný *casting*.
`c int i = 5; void *gen_ptr = &i; int val = *(int*)gen_ptr; // Casting nutný`

Ukazatelová aritmetika a vztah k polím (Array-pointer duality):

`p + 1` posune ukazatel o `sizeof(*p)` bajtů (tj. o velikost typu, na který ukazuje).

`pole[i]` je syntaktický cukr pro `*(pole + i)` .

C. Dynamická alokace paměti

Zásobník (Stack) vs. Halda (Heap)

Zásobník: Automatická správa, pro lokální proměnné/parametry. Velikost známá v kompilaci. Rychlá alokace/dealokace. Paměť zaniká s opuštěním bloku.

Halda: Manuální správa programátorem. Pro data s runtime velikostí nebo delší životností. Pomalejší alokace. Programátor musí explicitně uvolnit paměť.

Funkce pro dynamickou alokaci (`<stdlib.h>`)

`void* malloc(size_t size)` : Alokuje `size` bajtů. Paměť neiniculuje. Vrací `void*` nebo `NULL` při selhání. `c int *arr = (int*) malloc(10 * sizeof(int));`

`void* calloc(size_t num, size_t size)` : Alokuje `num * size` bajtů. **Vynuluje** paměť. Vrací `void*` nebo `NULL`. `c int *arr_zero = (int*) calloc(10, sizeof(int));`

`void* realloc(void* ptr, size_t new_size)` : Změní velikost bloku na `new_size`. Může přesunout data. Vrací `void*` nebo `NULL`. `c arr = (int*) realloc(arr, 20 * sizeof(int));`

`void free(void* ptr)` : Uvolní paměť alokovanou `malloc / calloc / realloc`. **Kriticky důležité** pro prevenci úniků paměti. `c free(arr);`

Správa paměti a rizika

Memory Leak: Dynamicky alokovaná paměť není uvolněna a program ztratí odkaz.

Dangling Pointer: Ukazatel míří na již uvolněnou paměť. Přístup vede k nedefinovanému chování. **Prevence:** Po `free(ptr);` vždy nastavit `ptr = NULL;`.

Double Free: Pokus o uvolnění již uvolněné paměti. Nedefinované chování.

Buffer Overflow: Zápis dat za hranice alokovaného bloku.

Kontrola `NULL` : Vždy kontrolovat návratové hodnoty alokačních funkcí.

Typické zkouškové otázky (zkrácené odpovědi)

Co je ukazatel a jak souvisí s adresou? Operátory `&` a `*`.

Ukazatel je proměnná, jejíž hodnota je paměťová adresa jiné proměnné.

`&` (adresa z): Získá adresu proměnné (např. `&x`).

`*` (dereference): Přistoupí k hodnotě na adrese, kam ukazatel ukazuje (např. `*p`).

Rozdíl mezi Stack a Heap? Použití `malloc` a `free`.

Stack: Automatická správa, lokální proměnné, kompilace-časová velikost, rychlé, paměť zaniká s funkcí.

Heap: Manuální správa, runtime velikost, delší životnost, pomalejší, programátor zodpovídá za `free()`.

`malloc(size)` : Alokuje `size` bajtů na haldě, neiniculuje, vrací `void*` nebo `NULL`.

`free(ptr)` : Uvolní paměť na haldě, na kterou ukazuje `ptr` .

"Array-pointer duality" a ukazatelová aritmetika s poli.

Jméno pole se ve většině kontextů "rozpadne" na ukazatel na svůj první prvek (`pole` se chová jako `&pole[0]`).

Ukazatelová aritmetika: `p + 1` posune ukazatel o `sizeof(*p)` bajtů (tj. o velikost typu, na který ukazuje).

`pole[i]` je ekvivalentní `*(pole + i)` .

Strukturované datové typy (`struct` , `union`) a jejich rozdíl.

`struct` : Seskupuje členy (různých typů), každý má vlastní paměť. Velikost je součet členů (+ padding). Všechny členy jsou platné současně.

`union` : Seskupuje členy, ale všichni sdílejí stejné paměťové místo. Velikost je rovna největšímu členu. V daný okamžik je platný jen jeden člen.

Rozdíl: `struct` = oddělená paměť; `union` = sdílená paměť.

"Memory Leak" a "Dangling Pointer" a prevence.

Memory Leak: Dynamicky alokovaná paměť není uvolněna a program ztratí odkaz. Vede k vyčerpání paměti.

Dangling Pointer: Ukazatel míří na již uvolněnou paměť. Přístup vede k nedefinovanému chování.

Prevence:

Memory Leak: Vždy uvolnit alokovanou paměť pomocí `free()` .

Dangling Pointer: Po `free(ptr);` vždy nastavit `ptr = NULL;` .

12. Algoritmy pro vyhledávání a řazení, složitost algoritmů

Klíčové pojmy

Algoritmus, časová složitost, paměťová složitost, Big-O notace, lineární vyhledávání, binární vyhledávání, quicksort, mergesort, heapsort, stabilita řazení, in-place algoritmus, pivot, halda.

A. Složitost algoritmů (Asymptotická složitost)

Porovnává efektivitu algoritmů nezávisle na hardwaru. Sleduje růst spotřeby zdrojů (čas/paměť) s velikostí vstupu n . * **Časová složitost:** Počet elementárních operací. *

Prostorová (paměťová) složitost: Množství dodatečné paměti.

Velká O notace (Asymptotická horní mez)

Značí $O(f(n))$, definuje asymptotickou horní mez pro růst funkce (nejhorší možný případ). Formálně: $g(n) \in O(f(n))$, pokud existují $c, n_0 > 0$ takové, že pro všechna $n \geq n_0$ platí $0 \leq g(n) \leq c \cdot f(n)$. Zanedbává aditivní konstanty a méně významné členy (např. $3n^2 + 5n + 10$ je $O(n^2)$).

Přehled základních tříd složitosti (od nejrychlejších)

$O(1)$ – **Konstantní:** Nezávisí na n . (Přístup k prvku pole dle indexu)

$O(\log n)$ – **Logaritmická:** Roste logaritmicky. (Binární vyhledávání)

$O(n)$ – **Lineární:** Roste přímo úměrně s n . (Lineární vyhledávání)

$O(n \log n)$ – **Linearitmetická:** Typická pro efektivní řadící algoritmy. (Merge Sort, Quick Sort (průměr))

$O(n^2)$ – **Kvadratická:** Roste s druhou mocninou n . (Bubble Sort, Insertion Sort)

$O(2^n)$ – **Exponenciální:** Extrémně náročné. (Naivní rekurzivní Fibonacci)

B. Algoritmy pro vyhledávání

Cíl: Najít pozici hledaného prvku.

1. Sekvenční (lineární) vyhledávání

Princip: Prochází pole prvek po prvku.

Podmínka: Data mohou být v libovolném pořadí.

Časová složitost: $O(n)$ (nejhorší i průměrný případ).

2. Binární vyhledávání (Metoda půlení intervalu)

Princip: Opakovaně dělí setříděné pole na poloviny, dokud prvek nenajde.

Podmínka: Data **musí být setříděná**.

Časová složitost: $O(\log n)$ (nejhorší i průměrný případ).

C. Algoritmy pro řazení (Třídění)

Cíl: Přeskládat prvky v poli podle klíče.

Vlastnosti algoritmů

Stabilita: Zachovává původní vzájemné pořadí prvků se stejnou hodnotou klíče.

In-place: Potřebuje jen konstantní množství dodatečné paměti $O(1)$ (kromě rekurzivního zásobníku).

1. Jednoduché algoritmy ($O(n^2)$)

Select Sort (Řazení výběrem): Najde min/max v neseříděné části, prohodí.

Vlastnosti: Nestabilní, In-place.

Složitost: $O(n^2)$ vždy.

Insert Sort (Řazení vkládáním): Bere prvek, vkládá na správné místo v seříděné části.

Vlastnosti: Stabilní, In-place.

Složitost: $O(n^2)$ (průměr/nejhorší), $O(n)$ (nejlepší – již seříděné).

Bubble Sort (Bublínkové řazení): Opakovaně porovnává a prohazuje sousední prvky.

Vlastnosti: Stabilní, In-place.

Složitost: $O(n^2)$ (průměr/nejhorší), $O(n)$ (nejlepší s optimalizací).

2. Pokročilé algoritmy ($O(n \log n)$)

Využívají strategii Rozděl a panuj (Divide and Conquer).

Quick Sort (Rychlé řazení): Zvolí **pivot**, přeuspořádá pole (menší vlevo, větší vpravo), rekurzivně aplikuje na podpole.

Vlastnosti: Nestabilní, In-place ($O(\log n)$ zásobník průměr, $O(n)$ nejhorší).

Složitost: $O(n \log n)$ (průměr), $O(n^2)$ (nejhorší – špatná volba pivotu).

Poznámka: V praxi často nejrychlejší.

Merge Sort (Řazení slučováním): Rekurzivně dělí pole na poloviny, pak seříděné úseky slučuje.

Vlastnosti: Stabilní, NENÍ In-place ($O(n)$ pomocné pole).

Složitost: $O(n \log n)$ vždy.

Poznámka: Vhodný pro externí řazení.

Heap Sort (Řazení haldou): Uspořádá prvky do binární haldy, opakovaně odebírá největší prvek z vrcholu.

Vlastnosti: Nestabilní, In-place.

Složitost: $O(n \log n)$ vždy.

Nejčastější chyby u zkoušky

Hodnocení algoritmu jen podle malého vstupu.

Použití binárního vyhledávání na neseřazená data.

Záměna stability a rychlosti řazení.

Neznalost rozdílu mezi časovou a paměťovou složitostí.

Neznalost dopadu volby pivotu u Quick Sortu.

Typické zkouškové otázky

Vysvětlete notaci Big-O.

Odpověď: Matematický zápis popisující asymptotickou horní mez růstu funkce (časové/paměťové složitosti) pro dostatečně velké n . Zanedbává konstanty a méně významné členy, zaměřuje se na dominantní člen.

Porovnejte quicksort a mergesort.

Quick Sort: Průměr $O(n \log n)$, nejhorší $O(n^2)$. In-place ($O(\log n)$ zásobník). Nestabilní. V praxi často rychlejší.

Merge Sort: Vždy $O(n \log n)$. Není In-place ($O(n)$ pomocná paměť). Stabilní. Vhodný pro externí řazení.

Kdy lze použít binární vyhledávání?

Odpověď: Pouze pokud jsou data, ve kterých se vyhledává, **striktně setříděná**.

Vysvětlete pojmy "stabilita" a "in-place" u řadicích algoritmů.

Stabilita: Algoritmus zachovává původní relativní pořadí prvků se stejnou hodnotou klíče.

In-place: Algoritmus potřebuje k řazení $O(1)$ dodatečné paměti (mimo rekurzivní zásobník).

Uveďte příklad algoritmu s $O(n^2)$ a $O(n \log n)$ časovou složitostí a vysvětlete, proč se liší.

$O(n^2)$: Např. **Bubble Sort**. Dva vnořené cykly vedou k $n \times n$ operacím.

$O(n \log n)$: Např. **Merge Sort**. Používá "rozděl a panuj": $\log n$ úrovní dělení, na každé úrovni $O(n)$ práce pro slučování. Efektivněji rozkládá problém.

13. Rekurse a zásobník programu (Tahák)

Klíčové pojmy

Rekurze, rekurzivní volání, báze rekurze, rekurzivní krok, zásobník programu (call stack), aktivační záznam (stack frame), návratová adresa, stack overflow, iterace, koncová rekurze, optimalizace koncové rekurze (TCO).

A. Pojem rekurze a její použití

Rekurze: Programovací technika, kdy se funkce volá sama (přímá/nepřímá).

Podmínky pro správnou rekurzi: * **Bázový (kotevní) případ:** Ukončovací podmínka, při které je výsledek znám a funkce se již nevolá rekurzivně. * **Rekurzivní krok:** Funkce volá sebe sama s modifikovaným vstupem, který se prokazatelně přibližuje k báзовému případu.

Typické použití rekurze: * **Hierarchické a stromové struktury:** Procházení stromů (In-order, Pre-order, Post-order), adresářů. * **Algoritmy Rozděl a panuj:** Quick Sort, Merge Sort, Binární vyhledávání. * **Kombinatorické úlohy:** Generování permutací, Hanojské věže, Backtracking, Fibonacciho posloupnost.

Výhody rekurze: Čitelnost, elegance, kratší kód pro rekurzivně definované problémy.

Nevýhody rekurze: Nižší výkon (režie volání funkcí), paměťová náročnost (zásobník), obtížnější ladění.

B. Využití zásobníku programu (Call Stack)

Zásobník programu (Call Stack): Datová struktura typu LIFO (Last-In, First-Out), spravuje volání funkcí. **Aktivační záznam (Stack Frame):** Vytvoří se na vrcholu zásobníku při každém volání funkce. Obsahuje: * Lokální proměnné * Parametry funkce * Návratovou adresu (kam se vrátit po dokončení funkce) * Uložené registry **Mechanismus:** Volání funkce → nový aktivační záznam na zásobník. Rekursivní volání → další záznam. Návrat z funkce → záznam se ze zásobníku odstraní (pop), řízení se vrátí na návratovou adresu.

Riziko: Stack Overflow (Přetečení zásobníku): Zásobník má omezenou velikost. Příliš hluboká rekurze (chybí báze, velký vstup) vyčerpá přidělenou paměť → pád programu.

C. Rekursivní vs. nerekursivní (iterativní) realizace

Každý rekurzivní algoritmus lze teoreticky převést na iterativní (pomocí cyklů a případně explicitního zásobníku na haldě).

Vlastnost	Rekurzivní realizace	Iterativní realizace
Čitelnost	Přirozenější pro rekurzivní problémy	Může být složitější
Výkon	Vyšší režie (volání funkcí, správa zásobníku)	Obvykle rychlejší
Paměť	Programový zásobník (stack), riziko stack overflow	Halda (heap) pro

Příklad: Výpočet faktoriálu $n!$ * **Rekurzivní:** `c long faktorial_rekurzivni(int n) { if (n <= 1) return 1; // Bázový případ return n * faktorial_rekurzivni(n - 1); // Rekurzivní krok }` Riziko Stack Overflow pro velké n . * **Iterativní:** `c long faktorial_iterativni(int n) { long vysledek = 1; if (n < 0) return -1; for (int i = 2; i <= n; i++) { vysledek *= i; } return vysledek; }` $O(1)$ paměť, robustnější.

Koncová rekurze (Tail Recursion): Speciální typ rekurze, kde je rekurzivní volání úplně poslední operací, kterou funkce provádí. **Optimalizace koncové rekurze (Tail Call Optimization - TCO):** Moderní kompilátory dokážou koncovou rekurzi optimalizovat. Místo vytváření nového aktivačního záznamu ji převedou na běžný cyklus, čímž eliminují režii volání a riziko Stack Overflow. Není garantováno ve všech jazycích/kompilátorech.

Typické zkouškové otázky

Definujte rekurzi a vysvětlete její základní principy. Uveďte příklad problému, který je přirozeně řešitelný rekurzí.

Rekurze: Funkce volá sama sebe.

Principy: **Bázový případ** (ukončení, známý výsledek), **Rekurzivní krok** (volání sebe s menším vstupem).

Příklad: Faktoriál ($n! = n \times (n - 1)!$, $0! = 1$), procházení stromové struktury.

Popište, jak rekurze využívá zásobník programu (call stack). Co je aktivační záznam a jaké informace obsahuje? Jaké riziko je s tímto mechanismem spojeno?

Při každém rekurzivním volání se na zásobník přidá nový **aktivační záznam (stack frame)**.

Aktivační záznam: Obsahuje lokální proměnné, parametry, návratovou adresu, uložené registry.

Riziko: Stack Overflow – příliš hluboká rekurze vyčerpá omezenou paměť zásobníku, program havaruje.

Vysvětlete rozdíl mezi rekurzivní a iterativní realizací algoritmu. Kdy je vhodnější použít jeden přístup před druhým?

Rekurzivní: Řeší problém voláním sebe sama, spoléhá na zásobník. Kód je často elegantnější pro rekurzivní problémy.

Iterativní: Řeší problém pomocí cyklů, explicitně spravuje stav. Obvykle paměťově efektivnější a rychlejší.

Kdy co: Rekurze pro přirozeně rekurzivní problémy (stromy, algoritmy rozděl a panuj). **Iterace** pro velké hloubky (prevence Stack Overflow), nebo když je kritický výkon a paměťová efektivita.

Co je koncová rekurze a jaká je její hlavní výhoda?

Koncová rekurze: Rekurzivní volání je **poslední operací** funkce.

Výhoda: Kompilátory mohou provést **Optimalizaci koncové rekurze (TCO)**, která ji převede na iterativní kód. Tím se eliminuje režie volání funkcí a riziko Stack Overflow, zachovává se elegance rekurzivního zápisu.

14. Členění programu v jazyce vyšší úrovně. Metody, funkce, procedury, makra. Parametry metod, procedur a funkcí a způsoby jejich předávání. Globální a lokální proměnné.

A. Členění programu (Modularita)

Dělení programu na menší, logicky ucelené celky pro: * Zvýšená přehlednost, čitelnost * Usnadnění testování * Zamezení duplicitě kódu (DRY) * Znovupoužitelnost, snadnější údržba * Abstrakce a skrývání detailů

Základní podprogramové jednotky: * **Funkce:** Blok kódu, který přijímá vstupy (parametry), provádí výpočet a **vrací hodnotu** (pomocí `return`). Pro výpočty, transformace dat. Ideálně bez vedlejších efektů. * **Procedura:** Blok kódu, který provádí posloupnost příkazů, ale **nevrací explicitně žádnou hodnotu** (v C/C++ má návratový typ `void`). Pro akce s vedlejšími efekty (např. I/O, změna globálních proměnných). * **Metoda:** Funkce, která je **přidružená k objektu nebo třídě** v objektově orientovaném programování (OOP). Volána na instanci objektu, má přístup k jeho datům (`this`). Klíčová pro zapouzdření. * **Makro:** Instrukce pro **preprocesor** (`#define`), která před kompilací provede **textovou substituci**. Není typově bezpečné. * **Nevýhody:** Není typově bezpečné, problémy s vyhodnocováním výrazů (chybějící závorky!), obtížné ladění, zvětšování kódu (Code Bloat). * **Alternativy (C++):** `inline` funkce, šablony.

B. Způsoby předávání parametrů

Formální parametry: Proměnné definované v hlavičce podprogramu.

Skutečné parametry (argumenty): Hodnoty/proměnné předané při volání.

Předávání hodnotou (Call by Value):

Princip: Do formálního parametru se **zkopíruje samotná hodnota** skutečného parametru.

Důsledek: Změny uvnitř podprogramu **neovlivní** původní proměnnou ve volajícím programu.

Kdy použít: Pro malé datové typy (int, char, float) nebo když nechceme, aby podprogram měnil původní data.

Předávání odkazem / referencí (Call by Reference):

Princip: Podprogramu se předá **odkaz na původní paměťové místo (adresa)**.

Důsledek: Změny uvnitř podprogramu se **okamžitě projeví na původní proměnné**.

Kdy použít: Když chceme, aby podprogram modifikoval původní data; pro efektivní předávání velkých datových struktur (objektů, polí) bez nákladného kopírování.

Realizace: V C++ přímá podpora referencí (`&`); v C se simuluje **předáním ukazatele (pointeru) hodnotou** (`*`).

Bezpečnost: `const` reference/ukazatele pro efektivní předávání bez možnosti modifikace.

Nejčastější chyby u zkoušky: Nepochopení rozdílu mezi hodnotou a odkazem, zapomenutí dereference ukazatele v C, předávání velkých struktur hodnotou, nezáměrná modifikace dat.

C. Globální a lokální proměnné (Rozsah platnosti a životní cyklus)

Rozsah platnosti (Scope): Oblast kódu, kde je proměnná přístupná.

Životní cyklus (Lifetime): Doba, po kterou proměnná existuje v paměti.

Lokální proměnné:

Definice: Deklarovány uvnitř funkce nebo bloku `{}` .

Rozsah platnosti: Viditelné a přístupné **pouze uvnitř této funkce/bloku**.

Životní cyklus: Vznikají na **zásobníku (Stack)** při vstupu do funkce/bloku a **automaticky zanikají** při opuštění.

Globální proměnné:

Definice: Deklarovány mimo jakoukoliv funkci.

Rozsah platnosti: Viditelné a modifikovatelné v **celém programu** (nebo v celém souboru, pokud jsou `static`).

Životní cyklus: Vznikají v **datovém segmentu paměti** při spuštění programu a existují po celou dobu jeho běhu. Implicitně inicializovány na nulu.

⚠ Rizika používání globálních proměnných (Anti-pattern):

Nečitelnost a obtížná údržba: Těžké sledovat změny.

Skryté závislosti (Side-effects): Funkce mají neočekávané vedlejší efekty.

Snížená znovupoužitelnost: Funkce jsou vázány na globální stav.

Problémy v multithreadingu (Race Conditions): Souběžný přístup bez synchronizace vede k nepředvídatelným výsledkům.

Ztížené testování: Nutnost nastavit/resetovat globální stav.

Alternativa: Předávání dat jako parametrů, OOP (zapouzdření).

Nejčastější chyby u zkoušky: Záměna rozsahu platnosti a životního cyklu, nadměrné používání globálních proměnných, předpoklad, že lokální proměnná si pamatuje

hodnotu mezi voláními, nepochopení inicializace.

Typické zkouškové otázky (Zkrácené odpovědi)

Vysvětlete rozdíl mezi funkcí, procedurou a metodou.

Funkce: Vrací hodnotu (`return`). Pro výpočty, transformace dat.

Procedura: Nevrací hodnotu (`void`). Pro akce s vedlejšími efekty (I/O, změna stavu).

Metoda: Funkce přidružená k objektu/třídě (OOP). Přístup k datům objektu (`this`).

Popište způsoby předávání parametrů (hodnotou a odkazem/referencí).

Předávání hodnotou: Předá se **kopie hodnoty**. Změny uvnitř funkce **neovlivní** originál. Pro malé typy, bez modifikace.

Předávání odkazem: Předá se **adresa paměti**. Změny uvnitř funkce **ovlivní** originál. Pro modifikaci dat, efektivní předávání velkých struktur.

Vysvětlete rozdíl mezi lokálními a globálními proměnnými z hlediska rozsahu platnosti a životního cyklu. Jaká jsou rizika používání globálních proměnných?

Lokální proměnné: Scope: Pouze uvnitř bloku. **Lifetime:** Vznikají na **Stacku** při vstupu, zanikají při opuštění.

Globální proměnné: Scope: Celý program/soubor. **Lifetime:** Vznikají v **datovém segmentu** při startu, existují po celou dobu běhu.

Rizika globálních: Nečitelnost, skryté závislosti (vedlejší efekty), **Race Conditions** (multithreading), snížená znovupoužitelnost, obtížné testování.

Co je makro a jaké jsou jeho hlavní nevýhody oproti funkcím?

Makro: Instrukce pro preprocesor, provádí **textovou substituci** v kódu.

Nevýhody: Není typově bezpečné, problémy s vyhodnocováním výrazů (závorky!), obtížné ladění, zvětšování kódu (Code Bloat).

Zde je zkrácená verze (tahák) k otázce "15. Objektově orientované programování":

15. Objektově orientované programování (OOP)

Klíčové pojmy

třída, objekt / instance

atribut, metoda

zapouzdření, dědičnost, polymorfismus

správa přístupu (`private` , `protected` , `public`)

abstraktní třída, rozhraní (interface)

genericita

kompozice

přetěžování metod (overloading), přepisování metod (overriding)

tabulka virtuálních metod (VMT)

A. Význam OOP

OOP organizuje program kolem **objektů**, které sdružují data (atributy) a související funkčnost (metody). * **Výhody:** Lepší modularita, znovupoužitelnost kódu, snadnější údržba. * **Třída:** Šablona/předpis pro objekty (definuje atributy a metody). * **Objekt / Instance:** Konkrétní zástupce třídy v paměti s vlastním stavem. * **Atributy:** Proměnné uvnitř třídy (stav objektu). * **Metody:** Funkce uvnitř třídy (chování objektu).

B. Tři základní pilíře OOP

Zapouzdření (Encapsulation):

Princip: Sdružení dat a metod do třídy. Skrytí vnitřního stavu objektu před vnějším světem.

Cíl: Ochrana integrity dat, snížení complexity.

Realizace: `private` atributy, přístup přes `public` Gettery a Settery.

Správa přístupu (Modifikátory viditelnosti):

`private` : Přístupné jen uvnitř třídy.

`protected` : Přístupné uvnitř třídy a v odvozených třídách.

`public` : Přístupné odkudkoliv.

Dědičnost (Inheritance):

Princip: Vytváření nových tříd (potomků) z existujících (rodičů). Potomek dědí atributy a metody, může je rozšiřovat/upravovat.

Cíl: Znovupoužitelnost kódu (DRY), tvorba logických hierarchií ("is-a" vztah).

Polymorfismus (Mnohotvárnost):

Princip: Schopnost různých objektů reagovat na stejné volání metody odlišným způsobem. Umožňuje jednotný přístup k objektům různých podtříd.

Cíl: Zvýšení flexibility a rozšiřitelnosti kódu.

Typy polymorfismu:

Statický (Compile-time): Přetěžování metod (Method Overloading): Více metod se stejným jménem, ale různými parametry.

Dynamický (Runtime): Přepisování metod (Method Overriding): Potomek předefinuje metodu rodiče. Rozhoduje se za běhu (využívá VMT).

Dědičnost vs. Kompozice

Dědičnost: "is-a" vztah. Silné provázání, problém "fragile base class".

Kompozice: "has-a" vztah. Objekt obsahuje jiné objekty. Volné provázání, flexibilita (chování lze měnit za běhu).

Doporučení: "Prefer composition over inheritance" pro větší flexibilitu a nižší provázanost.

C. Abstraktní třídy a Rozhraní (Interface)

Abstraktní třída:

Nelze instanciovat. Slouží jako základ pro dědění.

Může obsahovat abstraktní metody (bez implementace, povinně implementovat potomky) i běžné metody a atributy.

Vyjadřuje "is-a" vztah.

Rozhraní (Interface):

Čistý funkční kontrakt. Definuje sadu metod, které třída musí implementovat.

Tradičně obsahuje jen hlavičky metod a konstanty.

Třída rozhraní **implementuje**, nikoli dědí. Umožňuje vícenásobnou "dědičnost" chování. Vyjadřuje "can-do" vztah.

D. Genericita

Definice: Umožňuje psát kód (třídy/funkce) nezávisle na konkrétním datovém typu (typ jako parametr).

Cíl: Zvýšení znovupoužitelnosti a typové bezpečnosti (kompilátor kontroluje typy).

Využití: Datové kontejnery (např. `List<T>`), algoritmy.

Nejčastější chyby u zkoušky

Používání dědičnosti pro pouhé sdílení kódu (místo "is-a" vztahu).

`public` atributy místo `private` s Gettery/Settery (porušuje zapouzdření).

Zaměňování třídy (šablona) a objektu (konkrétní instance).

Typické zkouškové otázky

Vysvětlete zapouzdření, dědičnost a polymorfismus.

Zapouzdření: Sdružení dat/metod, skrytí vnitřního stavu, ochrana integrity dat.

Dědičnost: Nové třídy z existujících, "is-a" vztah, znovupoužitelnost.

Polymorfismus: Různé objekty reagují odlišně na stejné volání, flexibilita (přetěžování/přepisování metod).

Co je rozhraní? Kontrakt definující sadu metod k implementaci. Bez implementace (tradičně). Třída implementuje, nikoli dědí. Umožňuje vícenásobnou "dědičnost" chování.

Kdy dát přednost kompozici? Když je vztah "has-a" a vyžaduje se vysoká flexibilita a nízká provázanost (loose coupling), např. pro snadnou změnu chování za běhu.

16. Tvorba aplikací v prostředí konzole a MS Windows. Vývojová prostředí.

Klíčové pojmy

Konzolová aplikace (CLI), GUI aplikace, `stdin`, `stdout`, `stderr`, událostmi řízené programování, smyčka zpráv (Message Loop), zpráva (Message), procedura okna (Window Procedure / WndProc), Win32 API, frameworky (MFC, Qt, wxWidgets, Windows Forms, WPF), XAML, vývojové prostředí (IDE), textový editor, kompilátor, linker, build systém, debugger (breakpoints, krokování, sledování proměnných), profiler, verzovací systém.

16.1 Tvorba aplikací v prostředí konzole (CLI)

Aplikace v textovém režimu, komunikace textem. Tok řízení je **sekvenční a lineární**.

Princip fungování

Spustí se, vykonává instrukce, čeká na textový vstup, zpracuje, vypíše výstup, skončí po `main()`.

Vstup a výstup

Využívá standardní proudy: * `stdin` (standardní vstup): Klávesnice. C: `scanf()`, `getchar()`. * `stdout` (standardní výstup): Konzole. C: `printf()`. * `stderr` (standardní chybový výstup): Pro chybová hlášení. C: `fprintf(stderr, ...)`. Proudly lze přesměrovat (např. `program < input.txt > output.txt 2> errors.log`) nebo propojit rourami (`program1 | program2`).

Výhody

- Nízká režie, rychlý start.
- Snadná automatizace a skriptování.
- Jednodušší debugování (lineární tok).
- Zaměření na algoritmus.
- Multiplatformnost.

Nevýhody

- Omezená textová interakce.
- Není intuitivní pro běžné uživatele.

16.2 Tvorba aplikací v prostředí MS Windows (GUI)

Aplikace s grafickým rozhraním. Funguje na **událostmi řízeném programování**. Program čeká na události.

16.2.1 Architektura Win32 API a smyčka zpráv (Message Loop)

Zpráva (Message): Akce (klik, klávesa, překreslení) je OS převedena na strukturu `MSG` a vložena do fronty. Např. `WM_PAINT` , `WM_LBUTTONDOWN` .

Smyčka zpráv (Message Loop): Jádro GUI aplikace. Nekonečná smyčka:

`GetMessage()` : Blokuje, vybírá zprávu z fronty.

`TranslateMessage()` : Překládá virtuální klávesy na znakové vstupy (`WM_CHAR`).

`DispatchMessage()` : Předá zprávu příslušné proceduře okna.

Procedura okna (Window Procedure / WndProc): Callback funkce, volaná OS pro zpracování zpráv. Obsahuje `switch-case` pro reakci na `WM_PAINT` , `WM_LBUTTONDOWN` , `WM_DESTROY` atd.

Registrace a vytvoření okna: Nutné definovat (`WNDCLASSEX`) a zaregistrovat (`RegisterClassEx()`) třídu okna, pak vytvořit okno (`CreateWindowEx()`).

16.2.2 Moderní frameworky pro Windows

Zjednodušují vývoj nad Win32 API. * **C++:** * **MFC:** Starší objektový obal nad Win32 API. * **Qt:** Multiplatformní framework, vlastní signály/sloty. * **wxWidgets:**

Multiplatformní, snaží se o nativní vzhled. * **C# a .NET:** * **Windows Forms**

(WinForms): Starší, rychlý vývoj, pixelové pozicování, GDI+.

WPF (Windows Presentation Foundation): Moderní, odděluje vzhled (XAML) od logiky, DirectX, Data Binding, hardwarová akcelerace.

Výhody GUI

Intuitivní interakce (vizuální prvky).

Bohaté možnosti prezentace dat.

Vhodné pro komplexní uživatelské úlohy.

Nevýhody GUI

Vyšší režie, náročnější na zdroje.

Složitější vývoj a debugování (událostní tok).

Závislost na OS (nativní).

Potřeba asynchronního programování.

16.3 Vývojová prostředí (IDE - Integrated Development Environment)

Komplexní balík integrující nástroje pro efektivní tvorbu aplikací.

Základní součásti

Chytrý textový editor: Zvýrazňování syntaxe, automatické doplňování, refaktorování, navigace.

Kompilátor / Linker (Sestavovací nástroje):

Kompilátor: Překládá zdrojový kód na strojový kód.

Linker: Spojuje objektové soubory a knihovny do spustitelného programu.

Build systém: Spravuje proces sestavení projektu (např. Make, MSBuild).

Debugger (Ladicí nástroj): Pro hledání a opravu chyb.

Breakpoints: Zastaví vykonávání programu.

Krokování: `Step Over` (další řádek), `Step Into` (vstoupí do funkce), `Step Out` (dokončí funkci).

Sledování proměnných: Zobrazení aktuálních hodnot.

Zásobník volání (Call Stack): Posloupnost volaných funkcí.

Další integrované nástroje: Profiler, integrace s verzovacími systémy (Git), vizuální návrháře GUI.

Příklad relevantních IDE

Microsoft Visual Studio: Profesionální pro C++, C#, .NET.

JetBrains IDEs (CLion, Rider, IntelliJ IDEA, PyCharm): Cross-platformní, pokročilá analýza kódu.

Visual Studio Code (VS Code): Lehký, open-source, rozšiřitelný editor s funkcemi IDE přes pluginy.

Typické zkouškové otázky

Rozdíl mezi konzolovou a GUI aplikací:

CLI: Textová interakce (`stdin` / `stdout`). Tok řízení **sekvenční a lineární**.

GUI: Vizuální interakce (okna, tlačítka). Tok řízení **událostmi řízený**.

Princip událostmi řízeného programování a role smyčky zpráv:

Událostmi řízené programování: Program reaguje na události (uživatel, systém) spouštěním obslužných rutin.

Smyčka zpráv (Message Loop): Nekonečná smyčka, která: 1. Vybírá zprávy (`GetMessage()`). 2. Překládá zprávy (`TranslateMessage()`). 3. Předává zprávy `WndProc` (`DispatchMessage()`).

Hlavní komponenty IDE a k čemu slouží debugger:

Komponenty: Chytrý textový editor, Kompilátor/Linker, Debugger.

Debugger: Hledá a opravuje chyby. Umožňuje: spouštět v kontrolovaném režimu, nastavovat breakpoints, krokovat kód (`Step Over` / `Into` / `Out`), sledovat hodnoty proměnných, analyzovat zásobník volání.

Porovnání Windows Forms a WPF:

WinForms: Starší, GDI+, pixelové pozicování, slabší oddělení vzhledu/logiky. Pro rychlý vývoj jednodušších aplikací.

WPF: Modernější, DirectX, deklarativní XAML, silné oddělení (MVVM, Data Binding), hardwarová akcelerace. Pro bohatá a vizuálně komplexní UI.

Výhody a nevýhody konzolových aplikací:

Výhody: Nízká režie, rychlý start, snadná automatizace/skriptování, jednodušší debugování, multiplatformnost, zaměření na logiku.

Nevýhody: Omezená textová interakce, není intuitivní pro běžné uživatele, nevhodné pro vizuální úlohy.

Shrnutí kapitoly

Kapitola rozlišuje **konzolové (CLI)** aplikace (sekvenční tok, textová interakce přes standardní proudy) a **grafické (GUI)** aplikace (událostmi řízené programování, smyčka zpráv). Pro GUI se používají frameworky jako WPF (s XAML). Klíčovým nástrojem je **vývojové prostředí (IDE)**, které integruje editor, kompilátor a **debugger** pro efektivní vývoj a ladění.

Zde je zkrácená verze (tahák) zkouškové otázky:

17. Programátorské rozhraní operačního systému, vlákna, synchronizace

Klíčové pojmy

systémové volání (syscall), API, uživatelský režim (user space), režim jádra (kernel space), zpracování zpráv, fronta zpráv, událostní smyčka, proces, vlákno (thread), kontext procesu/vlákná, multithreading, sdílená paměť, zásobník (stack), ukazatel instrukcí (Instruction Pointer), race condition (souběh), kritická sekce, synchronizace, mutex, semafor (binary, counting), podmínková proměnná (condition variable), deadlock (uváznutí), starvation (hladovění), atomické operace, paměťové bariéry.

A. Programátorská rozhraní operačního systému (API / Syscalls)

Aplikace v **uživatelském prostoru (User Space)** nemají přímý přístup k HW. Žádají **jádro (Kernel)** o privilegované operace přes **API**, které volá **systémová volání (Syscalls)**. * **Syscall**: Mechanismus, kterým program v user space žádá jádro o službu. Zahrnuje **přepnutí režimu** (user ↔ kernel). * **Účel**: * **Bezpečnost**: Izolace aplikací. * **Abstrakce**: Jednotné rozhraní k HW. * **Správa zdrojů**: Jádro alokuje zdroje. * **Příklady**: POSIX (`fork()` , `open()`), Win32 API (`CreateProcess()` , `ReadFile()`). * **Kategorie**: Správa procesů, souborů, zařízení, informací, komunikace.

B. Zpracování zpráv (Message Processing / Event-Driven)

Základní technika pro asynchronní komunikaci a **programování řízené událostmi (Event-Driven Programming)**. * **Princip**: Odesílatel generuje **zprávy** do **fronty zpráv**. Příjemce v **událostní smyčce (Event Loop)** zprávy vybírá a zpracovává (pomocí callback funkcí). * **Fáze Event Loop**: Čekání na událost → Vybrání události → Zpracování události. * **Využití**: GUI aplikace, asynchronní I/O (Node.js), distribuované systémy.

C. Programování vláken (Multithreading)

Proces: Spuštěná instance programu. Vlastní **izolovaný paměťový prostor**, kontext (registry, paměťové mapy, soubory). Komunikace přes IPC.

Vlákno (Thread): Nejmenší jednotka vykonávání v rámci procesu. Vlákna **sdílejí paměťový prostor** procesu (halda, globální proměnné, kód), ale mají vlastní **zásobník (Stack)**, **ukazatel instrukcí (Instruction Pointer - IP)** a registry.

Typy:

User-Level Threads: Spravována knihovnou, jádro o nich neví. Rychlé přepínání, ale blokující volání blokuje celý proces.

Kernel-Level Threads: Spravována jádrem. Jádro plánuje, blokující volání neblokuje celý proces. (Většina moderních OS).

Stavy: Nový, Připravený, Běžící, Blokovaný/Čekající, Ukončený.

Výhody: Rychlejší přepínání kontextu, snadné sdílení dat, paralelismus, odezva.

Nevýhody: Složitost programování, Race Conditions, Deadlock, Starvation, obtížné ladění.

D. Synchronizace (Vláken a Procesů)

Race Condition (Souběh): Více vláken/procesů přistupuje ke **sdílenému zdroji** (alespoň jedno zapisuje) bez synchronizace. Výsledek závisí na náhodném proložení instrukcí.

Kritická sekce: Část kódu, která přistupuje ke sdílenému zdroji a musí být v jeden okamžik vykonávána maximálně jedním vláknem.

1. Mutex (Mutual Exclusion)

Princip: Digitální "zámek". Vlákno zamkne (`lock()`), vstoupí do kritické sekce, odemkne (`unlock()`). Pokud je zamčený, vlákno čeká. Zajišťuje **exkluzivní přístup** (max. jedno vlákno).

Pseudokód: `c lock(mutex); // Kritická sekce shared_counter++; unlock(mutex);`

2. Semafor

Princip: Zobecněný mutex s čítačem. Udává, kolik vláken může současně přistupovat ke zdroji.

`wait()` : Dekrementuje čítač. Pokud je záporný, vlákno se zablokuje.

`signal()` : Inkrementuje čítač. Probudí čekající vlákno.

Typy:

Binární semafor: Hodnota 0 nebo 1. Chová se jako mutex.

Čítací semafor: Omezení přístupu na konkrétní počet vláken (např. fond připojení).

3. Podmínkové proměnné (Condition Variables)

Princip: Slouží k signalizaci mezi vlákny čekajícími na splnění podmínky. Vždy s mutexem.

Vlákno čeká (`wait()`) na podmínku. Jiné vlákno splní podmínku a pošle signál (`signal()` - jedno, `broadcast()` - všechna).

Použití: Producent-Konzument problém.

Další synchronizační mechanismy:

Atomické operace: Hardwarově garantované nedělitelné operace (inkrementace, dekrementace).

Paměťové bariéry: Zajišťují pořadí operací s pamětí.

Klasické problémy synchronizace:

Deadlock (Uváznutí): Dvě+ vlákna/procesy čekají navzájem na prostředky, které drží to druhé.

Coffmanovy podmínky: Vzájemné vyloučení, Držení a čekání, Žádné preempce, Kruhové čekání.

Starvation (Hladovění): Vlákno nemůže získat přístup k požadovanému zdroji/procesoru kvůli neférovému plánování.

Problém spících filozofů: Klasický příklad deadlocku/starvation.

Praktické použití

OS API: Práce se soubory, správa procesů/vláken, síťová komunikace.

Zpracování zpráv: GUI, asynchronní I/O, IPC.

Vlákna a synchronizace: Paralelní výpočty, responzivní GUI, webové servery, databáze.

Nejčastější chyby u zkoušky

Záměna procesu a vlákna.

Ignorování režie systémových volání.

Nepochopení potřeby synchronizace.

Příliš široké kritické sekce.

Neznalost podmínek pro deadlock.

Záměna mutexu a semaforu.

Blokování hlavního GUI vlákna.

Typické zkouškové otázky

Co je systémové volání (syscall) a proč je potřeba?

Odpověď: Syscall je mechanismus, kterým user space program žádá kernel o privilegovanou službu (HW, I/O). Je potřeba pro bezpečnost a stabilitu OS, jelikož

aplikace nemají přímý přístup k systémovým zdrojům.

Vysvětlete rozdíl mezi procesem a vláknem.

Odpověď: **Proces** má vlastní izolovaný paměťový prostor a kontext. **Vláknem** je jednotka vykonávání v rámci procesu, sdílí paměť procesu (halda, globální), ale má vlastní zásobník, *IP* a registry. Přepínání vláken je rychlejší.

Co je kritická sekce a jaký je její vztah k race condition?

Odpověď: **Kritická sekce** je kód přistupující ke sdílenému zdroji, který smí vykonávat max. jedno vlákno. **Race condition** nastává, když více vláken přistupuje ke sdílenému zdroji bez synchronizace, což vede k nepředvídatelným výsledkům. Kritická sekce chrání před race condition.

Popište funkci mutexu a semaforu a uveďte, kdy byste který použili.

Odpověď: **Mutex** je binární zámek pro **exkluzivní přístup** (max. jedno vlákno) do kritické sekce. **Semafor** je zobecněný mutex s čítačem (`wait()` / `signal()`). **Binární semafor** je jako mutex. **Čítací semafor** omezuje přístup na **konkrétní počet** vláken současně (např. fond připojení).

Vysvětlete pojmy deadlock a starvation.

Odpověď: **Deadlock** je situace, kdy dvě+ vlákna čekají navzájem na prostředky, které drží to druhé (kruhové čekání). **Starvation** je situace, kdy vlákno nemůže získat zdroj/procesor po delší dobu kvůli neférovému plánování.

Jak funguje zpracování zpráv v GUI aplikacích?

Odpověď: Uživatelské/systémové události jsou zprávy ve **frontě zpráv**. Hlavní vlákno v **událostní smyčce** zprávy vybírá a předává **callbackům**, zajišťujíc tak asynchronní a responzivní GUI.

Shrnutí kapitoly

OS API/Syscalls zajišťují bezpečnost a správu zdrojů. Zpracování zpráv (event loop) je pro responzivní GUI a asynchronní I/O. Multithreading zvyšuje výkon, ale vyžaduje synchronizaci (mutexy, semaforey, podmínkové proměnné) k prevenci race conditions, deadlocků a starvation.

18. Operační systém (OS)

A. Vysvětlení pojmu

Operační systém (OS): Komplexní programové vybavení, **prostředník** mezi hardwarovými prostředky počítače a aplikačním softwarem.

Hlavní role: * **Správce prostředků:** Efektivně přiděluje a spravuje HW (CPU, paměť, I/O zařízení) mezi běžící programy. * **Rozšířený stroj:** Poskytuje aplikacím zjednodušenou a abstraktní reprezentaci hardwaru.

Kernel (jádro): Základní jádro OS, zavádí se při **bootování** do RAM. * **Privilegovaný režim (Kernel Mode):** Kernel běží s neomezeným přístupem k HW. *

Neprivilegovaný režim (User Mode): Běžné aplikace s omezenými oprávněními. Přístup k HW jen přes **systémová volání (Syscalls)** obsluhovaná jádrem. * **Cíl:** Zajištění bezpečnosti a stability systému.

B. Poskytované funkce (Služby OS)

Správa procesů a vláken:

Vytváření, plánování, synchronizace a ukončování procesů.

Plánovač (Scheduler): Přepíná procesy na CPU (multitasking).

Řeší synchronizaci (mutexy, semaforey), deadlock, race condition.

Správa paměti:

Přidělování a uvolňování RAM.

Izolace paměti: Procesy si nepřepisují paměť.

Virtuální paměť: Umožňuje aplikacím pracovat s větším adresním prostorem než fyzická RAM, pomocí **stránkování (paging)** a **swappingu** (přesun dat mezi RAM a diskem).

Správa souborů:

Abstrakce dat na úložištích do logické struktury adresářů a souborů.

Řízení přístupových práv.

Správa periferních zařízení (I/O):

Ovládání hardwaru pomocí **ovladačů (drivers)**.

Poskytuje aplikacím jednotné rozhraní.

Bezpečnost a ochrana: Autentizace, autorizace, ochrana před neoprávněným přístupem.

Uživatelské rozhraní:

CLI (Command Line Interface): Textové příkazy (Bash).

GUI (Graphical User Interface): Vizuální prvky (Windows Desktop).

C. Typy operačních systémů

1. Podle architektury jádra

Monolitické jádro:

Princip: Celé jádro (vč. ovladačů, FS) běží jako jeden celek v privilegovaném režimu.

Výhody: Vysoká rychlost.

Nevýhody: Nízká stabilita (selhání = pád celého jádra), obtížný vývoj.

Příklady: Linux, Unix.

Mikrojádro:

Princip: Minimum funkcí v privilegovaném režimu (správa adresního prostoru, plánování, meziprocesová komunikace – IPC). Ostatní jako uživatelské procesy.

Výhody: Vysoká stabilita, modularita.

Nevýhody: Pomalejší (režie IPC, Context Switch).

Příklady: QNX, MINIX.

Hybridní jádro:

Princip: Kombinace obou přístupů. Některé klíčové komponenty v privilegovaném režimu pro výkon.

Výhody: Snaží se spojit stabilitu s výkonem.

Příklady: Windows NT, macOS (XNU).

2. Podle účelu použití

OS pro PC a servery (General-Purpose OS): Vysoký výkon, uživatelský komfort, preemptivní multitasking, virtuální paměť. (Windows, Linux, macOS).

Operační systémy reálného času (RTOS): Determinizmus, garantované dokončení úloh v časovém limitu. **Hard/Soft Real-Time.** (QNX, FreeRTOS).

Síťové OS (Network OS): Správa síťových prostředků, sdílení. (Windows Server, Linux).

Distribuované OS (Distributed OS): Kolekce PC jako jeden systém, transparentnost. (Amoeba).

Embedded OS: Pro specifické účely, omezené zdroje, rychlý start. (FreeRTOS, Embedded Linux).

Mobilní OS: Optimalizovány pro dotykové rozhraní, napájení, konektivitu. (Android, iOS).

Typické zkouškové otázky

Definice OS, role, režimy:

OS: Prostředník mezi HW a SW.

Role: Správce prostředků (přiděluje HW), Rozšířený stroj (abstrahuje HW).

User Mode: Omezená oprávnění, aplikace.

Kernel Mode: Plný přístup k HW, jádro OS. Aplikace přes Syscalls.

4 hlavní funkce OS:

Správa procesů: Plánovač (multitasking).

Správa paměti: Virtuální paměť (stránkování, swapping).

Správa souborů: Souborový systém (organizace, práva).

Správa I/O: Ovladače (komunikace s HW).

Monolitické vs. Mikrojádro:

Monolitické: Vše v jádře (Linux). **Výhoda:** Rychlost. **Nevýhoda:** Nízká stabilita.

Mikrojádro: Minimum v jádře, zbytek jako uživatelské procesy (QNX). **Výhoda:** Stabilita. **Nevýhoda:** Pomalejší (IPC).

Virtuální paměť:

Co je: Umožňuje programu používat více paměti než fyzická RAM. Mapuje virtuální adresy na fyzické, přesouvá stránky na disk (stránkování/swapping).

Důležitost: Více programů najednou, izolace paměti, zjednodušuje programování.

RTOS:

Co je: Garantuje dokončení úloh v časovém limitu (**determinizmus**). Hard/Soft Real-Time.

Použití: Průmyslová automatizace, automobilový průmysl (ABS), avionika, lékařské přístroje, embedded systémy.

19. Správa procesů v operačním systému, vztah programu a procesu, životní cyklus procesu

Klíčové pojmy

Proces, stav procesu, PCB (Process Control Block), plánování, kontext, fork, exec, zombie proces.

A. Vztah programu a procesu

Program (Pasivní entita): Statický soubor na disku (např. `.exe`), obsahuje instrukce a statická data.

Proces (Aktivní entita): Spuštěná instance programu v RAM. Má přidělené systémové prostředky (paměť, CPU čas, soubory).

Příklad: Jeden program Chrome na disku, tři otevřená okna = tři samostatné procesy v RAM.

B. Struktura procesu v paměti

Adresní prostor procesu se dělí na: * **Textový segment (Code):** Strojové instrukce, Read-Only. * **Datový segment (Data):** Globální a statické proměnné (inicializované/neinicializované). * **Halda (Heap):** Dynamicky alokovaná paměť za běhu (*malloc*, *new*), roste nahoru. * **Zásobník (Stack):** Lokální proměnné, parametry funkcí, návratové adresy, roste dolů.

C. Řízení procesu: PCB (Process Control Block)

Datová struktura v jádře OS, udržuje informace o procesu: * **PID:** Unikátní ID procesu. * **Stav procesu:** Aktuální fáze životního cyklu. * **Kontext procesoru:** Uložené hodnoty registrů CPU (vč. Instruction Pointeru a Stack Pointeru). * **Informace o správě paměti:** Ukazatele na tabulky stránek. * **Vlastník a oprávnění:** Uživatel, oprávnění. * **Seznam otevřených souborů a prostředků:** Deskriptory souborů, socketů.

D. Životní cyklus a stavy procesu

Nový (New): Proces se vytváří, OS alokuje paměť, zakládá PCB.

Připravený (Ready): Má prostředky, čeká na přidělení CPU plánovačem.

Běžící (Running): Procesor vykonává jeho instrukce. Na jednom jádru CPU jen jeden proces.

Čekající / Blokovaný (Waiting / Blocked): Čeká na externí událost (I/O, stisk klávesy, uvolnění mutexu). Nespotřebovává CPU.

Ukončený (Terminated / Zombie): Dokončil práci nebo byl ukončen. Prostředky uvolněny, PCB zůstává, dokud rodič nepřechte návratový kód (zombie).

Přechody mezi stavy: * **Ready** → **Running**: Plánovač vybere proces (Dispatch). *

Running → **Ready**: Vypršel časový kvant (preempce) nebo vyšší priorita. *

Running → **Waiting**: Proces zažádal o I/O operaci nebo se zablokoval. *

Waiting → **Ready**: I/O operace dokončena nebo událost nastala. *

Running → **Terminated**: Proces dokončil práci nebo byl ukončen.

E. Přepínání kontextu (Context Switch)

Mechanismus pro multitasking, umožňuje střídání procesů na CPU: 1. OS pozastaví proces A. 2. Uloží registry CPU do PCB procesu A. 3. Načte registry z PCB procesu B do CPU. 4. Změní mapování virtuální paměti na adresní prostor B. 5. Procesor pokračuje v B. *

Režie: Čistá výpočetní režie, CPU nevykonává užitečný kód aplikace.

Praktické použití

Multitasking, izolace aplikací, správa prostředků, stabilita systému.

Nejčastější chyby u zkoušky

Zaměňování programu a procesu.

Neuvědomění si režie přepínání kontextu.

Neznalost stavu *Waiting* při I/O.

Typické zkouškové otázky

Popište životní cyklus procesu.

Odpověď: Zahrnuje stavy Nový (vytváření), Připravený (čeká na CPU), Běžící (vykonává na CPU), Čekající (čeká na I/O/událost) a Ukončený (dokončil/ukončen). Přechody řídí plánovač, I/O operace nebo dokončení činnosti.

Co obsahuje PCB?

Odpověď: PCB je datová struktura v jádře OS obsahující PID, stav procesu, kontext procesoru (registry), info o správě paměti, vlastníka/oprávnění a seznam otevřených prostředků.

Jak se liší proces a vlákno?

Odpověď: Proces je samostatná instance programu s vlastním adresním prostorem a prostředky (izolace). Vlákno je lehčí jednotka vykonávání v rámci jednoho procesu,

sdílí adresní prostor a prostředky procesu, ale má vlastní zásobník, registry a program counter (souběžnost s menší režii).